

Introduction to Deep Learning

Session 7: Generalization & Regularization

May 2026

Magda Gregorová - magda.gregorova@thws.de



Licensed according to CC-BY-SA 4.0

Lecture Overview

1. Generalization

2. The Generalization Paradox

3. Regularisation Techniques

Where We Are

So far we have:

- **Network** $f_{\theta} : \mathcal{X} \rightarrow \mathcal{Y}$ - parameterized function
- **Loss** $\mathcal{L}(\theta; \mathcal{D})$ - measures fit on training data
- **Optimization** - algorithms to minimize $\mathcal{L}(\theta; \mathcal{D})$

Open question:

Does minimizing training loss give us a **good model**?

This lecture: what good actually means, and how to get there

What we optimize:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(\mathbf{x}^{(i)}), y^{(i)})$$

What we actually want: low loss on **new, unseen examples from the population**

$$\mathcal{L}_{\text{pop}}(\theta) = \mathbb{E}_{(\mathbf{x}, y) \sim p_{\text{data}}} [\ell(f_{\theta}(\mathbf{x}), y)]$$

Generalization gap:

$$\underbrace{\mathcal{L}_{\text{pop}}(\theta)}_{\text{true goal}} - \underbrace{\mathcal{L}_{\text{train}}(\theta)}_{\text{what we minimize}} \geq 0$$

The **test set** is a finite sample we create to *estimate* $\mathcal{L}_{\text{pop}}$ — it is a measurement tool, not the objective itself

Train / Validation / Test Split

Training set

Fit model parameters

$$\min_{\theta} \mathcal{L}(\theta; \mathcal{D}_{\text{train}})$$

Validation set

Select model, tune
hyperparameters

$$\mathcal{L}(\theta; \mathcal{D}_{\text{val}})$$

Touched many times during
development

Test set

Final honest evaluation

$$\mathcal{L}(\theta; \mathcal{D}_{\text{test}})$$

Touched once, at the end

Common mistake: tuning on the test set - reported performance is optimistic

Typical splits: 80/10/10 or 70/15/15 - depends on dataset size

Overfitting and Underfitting

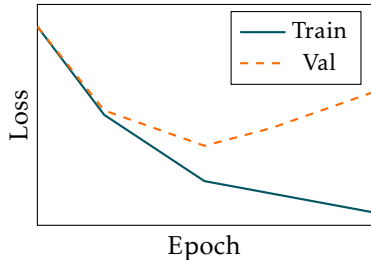
Underfitting: model too simple

- High train loss, high test loss
- Small generalisation gap

Overfitting: model memorises training data

- Low train loss, high test loss
- **Large generalisation gap**

Signal: monitor the generalisation gap — train loss alone tells us nothing about generalisation



What To Do

Underfitting (high train loss, small gap)

- Increase model capacity
- Train longer
- Tune learning rate and optimiser
- Reduce regularisation if applied

Overfitting (low train loss, large gap)

- More data — always most effective
- **Regularisation** — subject of this lecture
- Reduce model capacity

This lecture: deep networks have enormous capacity yet often generalise well — we study why, and how to ensure it

Classical view: test error decomposes into two competing terms

Bias — error from wrong assumptions

- Model too simple to capture the data
- High train loss, high test loss
- Underfitting

Variance — sensitivity to training set

- Model fits noise in training data
- Low train loss, high test loss
- Overfitting

Classical prescription: choose model complexity to balance the two

More parameters $\Rightarrow$ lower bias, higher variance $\Rightarrow$ **overfitting**

Classical prediction: more parameters than training examples $\Rightarrow$ catastrophic overfitting

Reality in deep learning:

- Deep networks have huge numbers of parameters — far more than needed to memorise training data
- They routinely drive training loss close to zero
- Classical theory: a model that nearly memorises training data cannot generalise
- In practice: deep networks still generalise well

Why do deep networks generalise despite being massively overparameterised?

Many Good Minima

Classical concern: gradient descent finds a local minimum — but which one?

Empirical observation in deep networks:

- Overparameterised networks have a vast number of local minima
- Most local minima have very similar loss values
- Networks trained from different initialisations reach different minima — yet generalise similarly

Why: the network has far more parameters than needed to fit the data — many equivalent solutions exist

Which minimum we find matters much less than classical theory suggests

Flat vs Sharp Minima

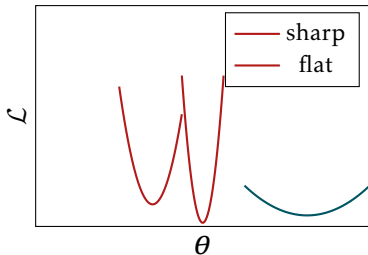
Not all minima generalise equally well

Sharp minimum:

- Loss changes rapidly
- Small distribution shift $\Rightarrow$ loss jumps
- Poor generalisation

Flat minimum:

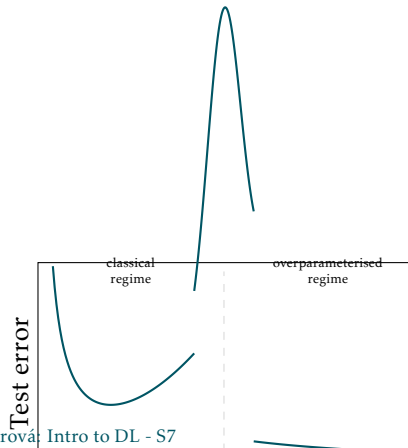
- Loss changes slowly
- Small distribution shift $\Rightarrow$ loss stays low
- Good generalisation



SGD noise naturally favours flat minima - implicit regularisation from mini-batching
(Session 5)

Classical bias-variance curve: test error has one optimal complexity — U-shaped curve

Double descent: beyond the interpolation threshold, test error decreases again



- **Underfitting:** too little capacity
- **Interpolation threshold:** maximally rigid — worst generalisation
- **Overparameterised:** smoother interpolation — generalisation recovers

Deep networks operate in the overparameterised regime

Section 4

Regularisation Techniques

Explicit regularisation: add penalty term to the loss

$$\tilde{\mathcal{L}}(\theta) = \mathcal{L}(\theta) + \frac{\lambda}{2} \|\theta\|^2$$

Large weights $\Rightarrow$ model relies heavily on specific features $\Rightarrow$ high variance

L2 regularisation (weight decay)

- Penalises $\|\theta\|^2$ — shrinks all weights
- Smooth, differentiable
- `weight_decay` in `torch.optim`

L1 regularisation

- Penalises $\|\theta\|_1$ — promotes sparsity
- Some weights go exactly to zero
- Useful for feature selection

Hyperparameter: $\lambda > 0$ controls strength — tuned on validation set

In deep learning: useful baseline, freely combined with other techniques — but less central than in classical ML

Dropout

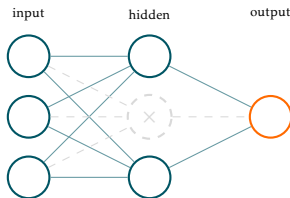
During training: randomly zero each activation with probability p

- Each forward pass uses a different subnetwork
- Neurons cannot rely on specific others always being present
- Prevents co-adaptation of neurons

During evaluation: all neurons active

- Outputs scaled by $(1 - p)$ to compensate
- Equivalently: weights multiplied by $(1 - p)$ at test time

Typical values: $p = 0.1-0.5$ — tuned on validation set



Why does dropout work?

With n neurons, dropout implicitly trains 2^n different subnetworks — one per possible dropout mask

- Each mini-batch trains a different subnetwork
- Subnetworks share weights — efficient training
- At test time: full network approximates **averaging** over all subnetworks

Ensembles generalise better than single models — averaging reduces variance

Dropout achieves ensemble-like generalisation at the cost of a single model

`model.train()` and `model.eval()`

Some layers behave differently during training and evaluation:

Dropout

- **Train:** randomly zero activations with prob p
- **Eval:** all neurons active, outputs scaled by $(1 - p)$

Batch Normalisation

- **Train:** normalise using mini-batch statistics
- **Eval:** normalise using running statistics accumulated during training

All other layers — `nn.Linear`, `nn.ReLU`, etc. — are unaffected by the mode

Always call `model.eval()` before validation and inference

Always call `model.train()` before resuming training

A recurring theme: many effective regularisation techniques inject randomness

Technique	Source of randomness	Type
Weight decay	—	explicit, deterministic
Mini-batch SGD	random batch selection	implicit
Dropout	random neuron masking	explicit
Data augmentation	random transformations	explicit

Mini-batch SGD as implicit regularisation:

- No change to the loss function
- Gradient noise prevents convergence to sharp minima
- Flat minima found for free — at no extra cost